

Porter - NNRegression

Importing Libraries

When an order is placed, predict how many minutes it will take to deliver the order to the customer. Accurate delivery-time prediction can help a food-delivery platform improve ETA estimates, customer experience, delivery-partner allocation, and operational planning.

```
In [47]: # Libraries to analyze data
import numpy as np
import pandas as pd

# Libraries to visualize data
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.neighbors import LocalOutlierFactor

from sklearn.impute import KNNImputer

from sklearn.preprocessing import OneHotEncoder
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from statsmodels.stats.outliers_influence import variance_inflation_factor
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Activation, LeakyReLU, Dropout, BatchNormalization
from tensorflow.keras.optimizers import Adam, RMSprop
from sklearn.metrics import mean_squared_error
```

Upload dataset

```
In [48]: df = pd.read_csv('Porter_data_2.csv')
df
```

Out[48]:

	market_id	created_at	actual_delivery_time	store_primary_category	order_protocol
0	1	2/6/2015 22:24	2/6/2015 23:11	4	1
1	2	2/10/2015 21:49	2/10/2015 22:33	46	2
2	2	2/16/2015 0:11	2/16/2015 1:06	36	3
3	1	2/12/2015 3:36	2/12/2015 4:35	38	1
4	1	1/27/2015 2:12	1/27/2015 2:58	38	1
...	...	...	...	...	...
175772	1	2/17/2015 0:19	2/17/2015 1:02	28	4
175773	1	2/13/2015 0:01	2/13/2015 1:03	28	4
175774	1	1/24/2015 4:46	1/24/2015 5:32	28	4
175775	1	2/1/2015 18:18	2/1/2015 19:03	58	1
175776	1	2/8/2015 19:24	2/8/2015 20:01	58	1

175777 rows × 14 columns



In [49]:

```
df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 175777 entries, 0 to 175776
Data columns (total 14 columns):
 #   Column                                     Non-Null Count  Dtype
---  -
 0   market_id                               175777 non-null  int64
 1   created_at                              175777 non-null  object
 2   actual_delivery_time                    175777 non-null  object
 3   store_primary_category                  175777 non-null  int64
 4   order_protocol                          175777 non-null  int64
 5   total_items                             175777 non-null  int64
 6   subtotal                                175777 non-null  int64
 7   num_distinct_items                      175777 non-null  int64
 8   min_item_price                          175777 non-null  int64
 9   max_item_price                          175777 non-null  int64
10   total_onshift_dashers                   175777 non-null  int64
11   total_busy_dashers                      175777 non-null  int64
12   total_outstanding_orders                175777 non-null  int64
13   estimated_store_to_consumer_driving_duration 175777 non-null  int64
dtypes: int64(12), object(2)
memory usage: 18.8+ MB

```

```
In [50]: df.shape
```

```
Out[50]: (175777, 14)
```

```
In [51]: df.isna().sum()
```

```

Out[51]: market_id                0
         created_at               0
         actual_delivery_time      0
         store_primary_category    0
         order_protocol            0
         total_items               0
         subtotal                  0
         num_distinct_items        0
         min_item_price            0
         max_item_price            0
         total_onshift_dashers     0
         total_busy_dashers        0
         total_outstanding_orders  0
         estimated_store_to_consumer_driving_duration 0
dtype: int64

```

```
In [52]: df.nunique()
```

```
Out[52]: market_id          6
         created_at        23362
         actual_delivery_time 23420
         store_primary_category 73
         order_protocol      7
         total_items         54
         subtotal            8182
         num_distinct_items   20
         min_item_price       2251
         max_item_price       2585
         total_onshift_dashers 172
         total_busy_dashers   158
         total_outstanding_orders 281
         estimated_store_to_consumer_driving_duration 1318
         dtype: int64
```

```
In [53]: df.duplicated().value_counts()
```

```
Out[53]: False    175776
         True      1
         Name: count, dtype: int64
```

```
In [54]: df.describe()
```

```
Out[54]:
```

	market_id	store_primary_category	order_protocol	total_items	subtotal
count	175777.000000	175777.000000	175777.000000	175777.000000	175777.000000
mean	2.743726	35.887949	2.911752	3.204976	2697.111147
std	1.330963	20.728254	1.513128	2.674055	1828.554893
min	1.000000	0.000000	1.000000	1.000000	0.000000
25%	2.000000	18.000000	1.000000	2.000000	1412.000000
50%	2.000000	38.000000	3.000000	3.000000	2224.000000
75%	4.000000	55.000000	4.000000	4.000000	3410.000000
max	6.000000	72.000000	7.000000	411.000000	26800.000000

```
In [55]: df.describe(include='object')
```

```
Out[55]:
```

	created_at	actual_delivery_time
count	175777	175777
unique	23362	23420
top	2/11/2015 19:51	2/14/2015 3:57
freq	48	42

```
In [56]: min(df['created_at']), max(df['created_at'])
```

```
Out[56]: ('1/21/2015 15:22', '2/9/2015 6:00')
```

There are 175777 entries with 14 columns The data is available between 21-Jan-2015 to 9-Feb-2015, 2 months of data There are no null/missing values in each of the dates There is only 1 duplicate value The columns market_id, total_onshift_partners, total_busy_partners and total_outstanding_orders can be of type int64 The columns created_at and actual_delivery_time need to be of type datetime Extract hour and day of the order placement from created_at Create delivery_time_mins column by subtracting created_at from actual_delivery_time Columns created_at and actual_delivery_time can be dropped

```
In [57]: ## Removing 1 duplicate value
```

```
df = df.drop_duplicates().reset_index(drop=True)

print(df.shape)
print(df.duplicated().sum())
```

```
(175776, 14)
```

```
0
```

```
In [58]: # 'created_at' and 'actual_delivery_time' converted to of type datetime
```

```
df[['created_at', 'actual_delivery_time']] = df[['created_at', 'actual_delivery_time']].astype('datetime64[ns]')
df[['market_id', 'order_protocol', 'num_distinct_items']] = df[['market_id', 'order_protocol', 'num_distinct_items']].astype('category')
```

```
In [59]: df['created_hour'] = df['created_at'].dt.hour # created - created_hour
df['created_day'] = df['created_at'].dt.dayofweek # created - created_day
df['delivery_time_mins'] = round((df['actual_delivery_time'] - df['created_at']) / pd.Timedelta('1min')).astype('int64')
df.drop(columns=['created_at', 'actual_delivery_time'], inplace=True) # dropping of
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 175776 entries, 0 to 175775
Data columns (total 15 columns):
```

#	Column	Non-Null Count	Dtype
0	market_id	175776 non-null	category
1	store_primary_category	175776 non-null	int64
2	order_protocol	175776 non-null	category
3	total_items	175776 non-null	int64
4	subtotal	175776 non-null	int64
5	num_distinct_items	175776 non-null	category
6	min_item_price	175776 non-null	int64
7	max_item_price	175776 non-null	int64
8	total_onshift_dashers	175776 non-null	int64
9	total_busy_dashers	175776 non-null	int64
10	total_outstanding_orders	175776 non-null	int64
11	estimated_store_to_consumer_driving_duration	175776 non-null	int64
12	created_hour	175776 non-null	int32
13	created_day	175776 non-null	int32
14	delivery_time_mins	175776 non-null	float64

```
dtypes: category(3), float64(1), int32(2), int64(9)
```

```
memory usage: 15.3 MB
```

```
In [60]: df.describe()
```

Out[60]:

	store_primary_category	total_items	subtotal	min_item_price	max_item_pri
count	175776.000000	175776.000000	175776.000000	175776.000000	175776.0000
mean	35.888017	3.204988	2697.122224	684.967060	1160.1629
std	20.728294	2.674058	1828.554196	519.883955	560.8272
min	0.000000	1.000000	0.000000	-86.000000	0.0000
25%	18.000000	2.000000	1412.000000	299.000000	799.0000
50%	38.000000	3.000000	2224.000000	595.000000	1095.0000
75%	55.000000	4.000000	3410.000000	942.000000	1395.0000
max	72.000000	411.000000	26800.000000	14700.000000	14700.0000

In []:

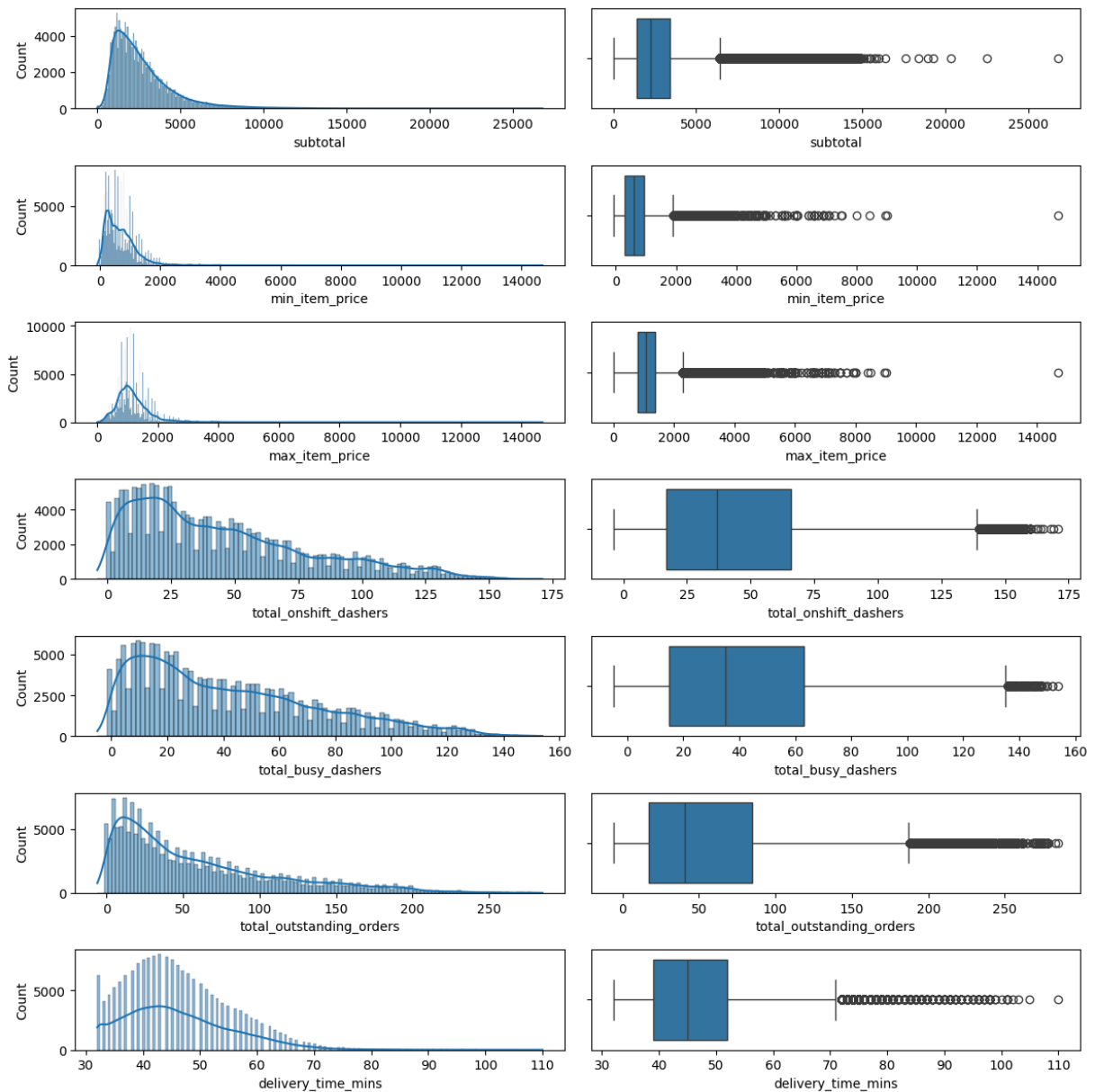
Exploratory Data Analysis

1. Univariate Analysis

```

In [61]: fig, axes = plt.subplots(nrows=7, ncols=2, figsize = (12, 12))
sns.histplot(data=df, x = "subtotal", kde=True, ax=axes[0,0])
sns.boxplot(data=df, x = "subtotal", ax=axes[0,1])
sns.histplot(data=df, x = "min_item_price", kde=True, ax=axes[1,0])
sns.boxplot(data=df, x = "min_item_price", ax=axes[1,1])
sns.histplot(data=df, x = "max_item_price", kde=True, ax=axes[2,0])
sns.boxplot(data=df, x = "max_item_price", ax=axes[2,1])
sns.histplot(data=df, x = "total_onshift_dashers", kde=True, ax=axes[3,0])
sns.boxplot(data=df, x = "total_onshift_dashers", ax=axes[3,1])
sns.histplot(data=df, x = "total_busy_dashers", kde=True, ax=axes[4,0])
sns.boxplot(data=df, x = "total_busy_dashers", ax=axes[4,1])
sns.histplot(data=df, x = "total_outstanding_orders", kde=True, ax=axes[5,0])
sns.boxplot(data=df, x = "total_outstanding_orders", ax=axes[5,1])
sns.histplot(data=df, x = "delivery_time_mins", kde=True, ax=axes[6,0])
sns.boxplot(data=df, x = "delivery_time_mins", ax=axes[6,1])
plt.tight_layout()
plt.show()

```

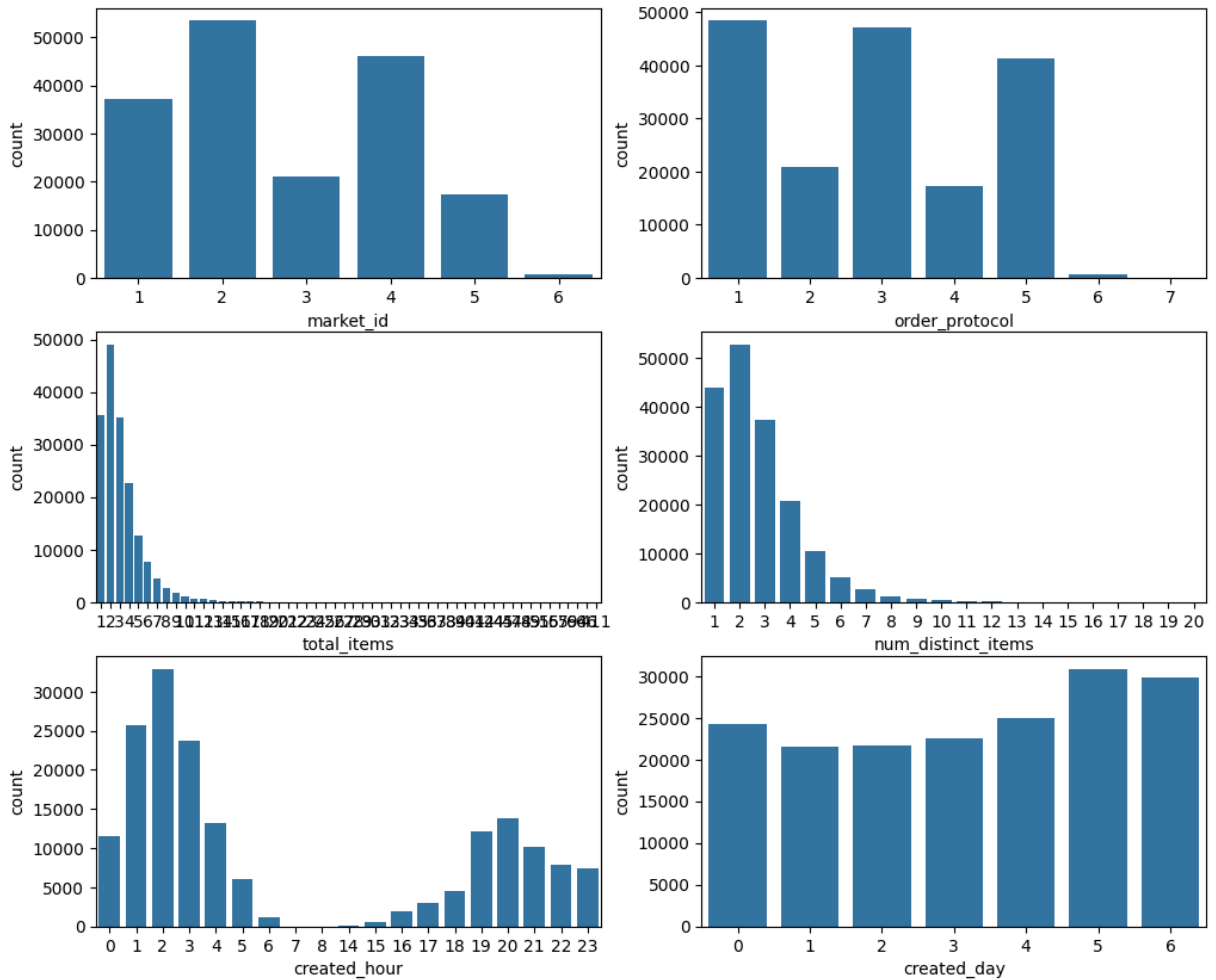


Majority of the subtotal is in the range of 0 to 5000. Majority of min_item_price and max_item_price are in the range of 0 to 2000. total_onshift_partners, total_busy_dashers, total_outstanding_dashers and delivery_time_mins seem to follow similar kind of distribution - right skewed. The boxplot clearly shows the presence of outliers in subtotal, min_item_price, max_item_price and delivery_time_mins.

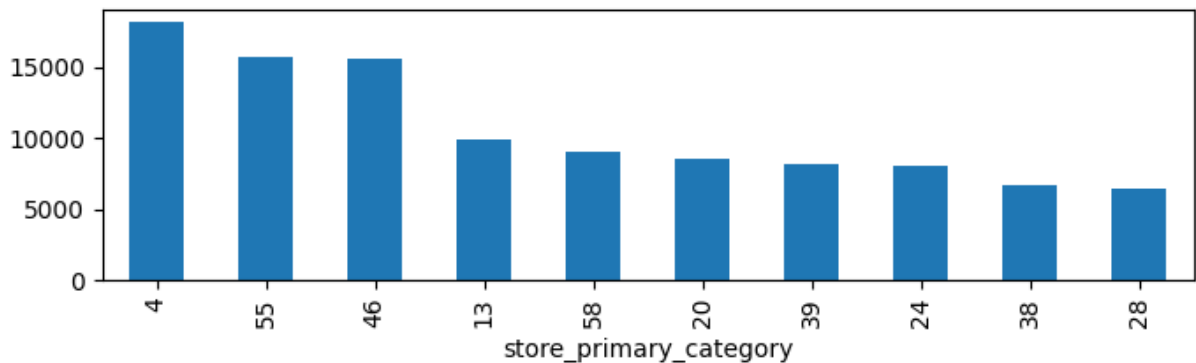
In []:

Categorical variables

```
In [62]: fig, axes = plt.subplots(3,2,figsize=(12,10))
sns.countplot(ax=axes[0,0], data=df, x='market_id')
sns.countplot(ax=axes[0,1], data=df, x='order_protocol')
sns.countplot(ax=axes[1,0], data=df, x='total_items')
sns.countplot(ax=axes[1,1], data=df, x='num_distinct_items')
sns.countplot(ax=axes[2,0], data=df, x='created_hour')
sns.countplot(ax=axes[2,1], data=df, x='created_day')
plt.show()
```



```
In [65]: df['store_primary_category'].value_counts()[:10].plot(kind='bar', figsize = (8, 2))
plt.show()
```



market_id 2 is the major contributor Majority of order placement are through protocol 1 and 3 Majority of orders have 2 num_distinct_items as well as 2 total_items Majority of the orders are placed at around 2AM and on weekends Majority of orders delivered are from restaurant 4

```
In [66]: mask = (df['min_item_price'] >= 0) & (df['max_item_price'] >= 0) & (df['total_onshi'] >= 0)
df = df[mask]
```

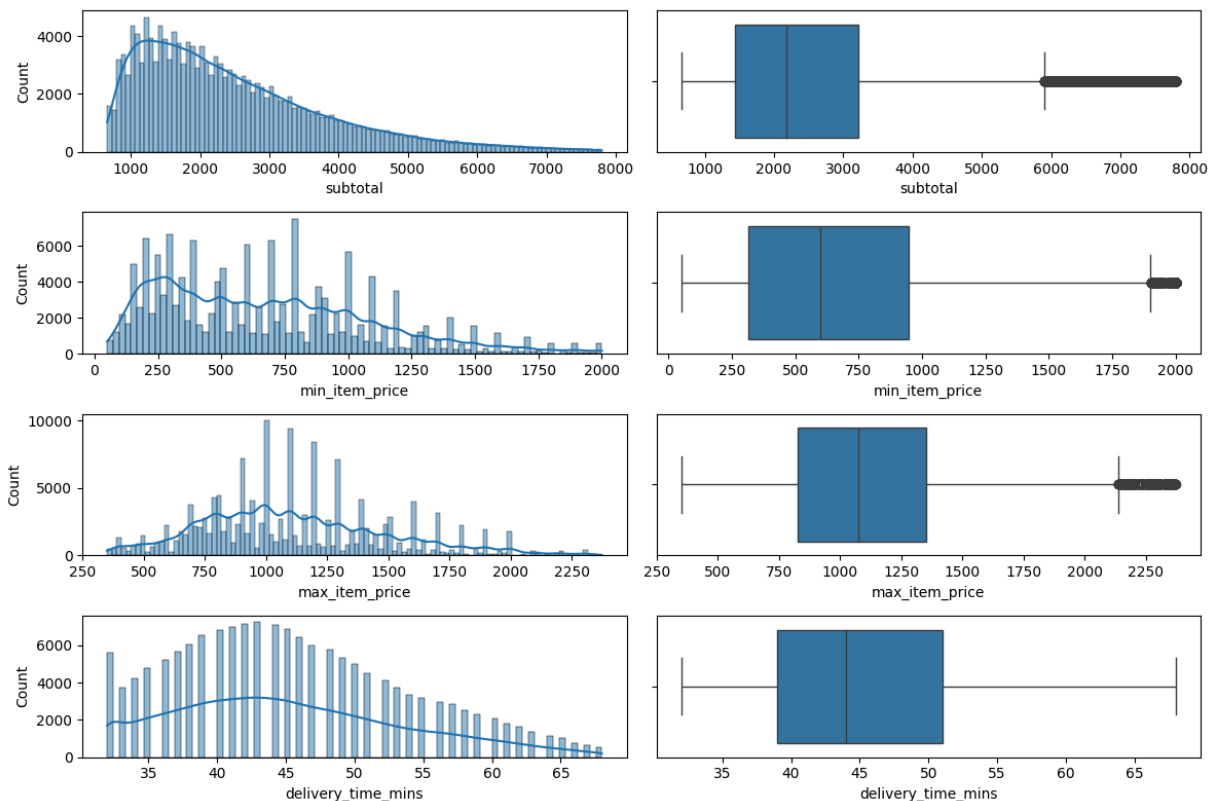
Right-skewed distributions and outliers, particularly in order value, item price, and delivery time.

Handling Outlier

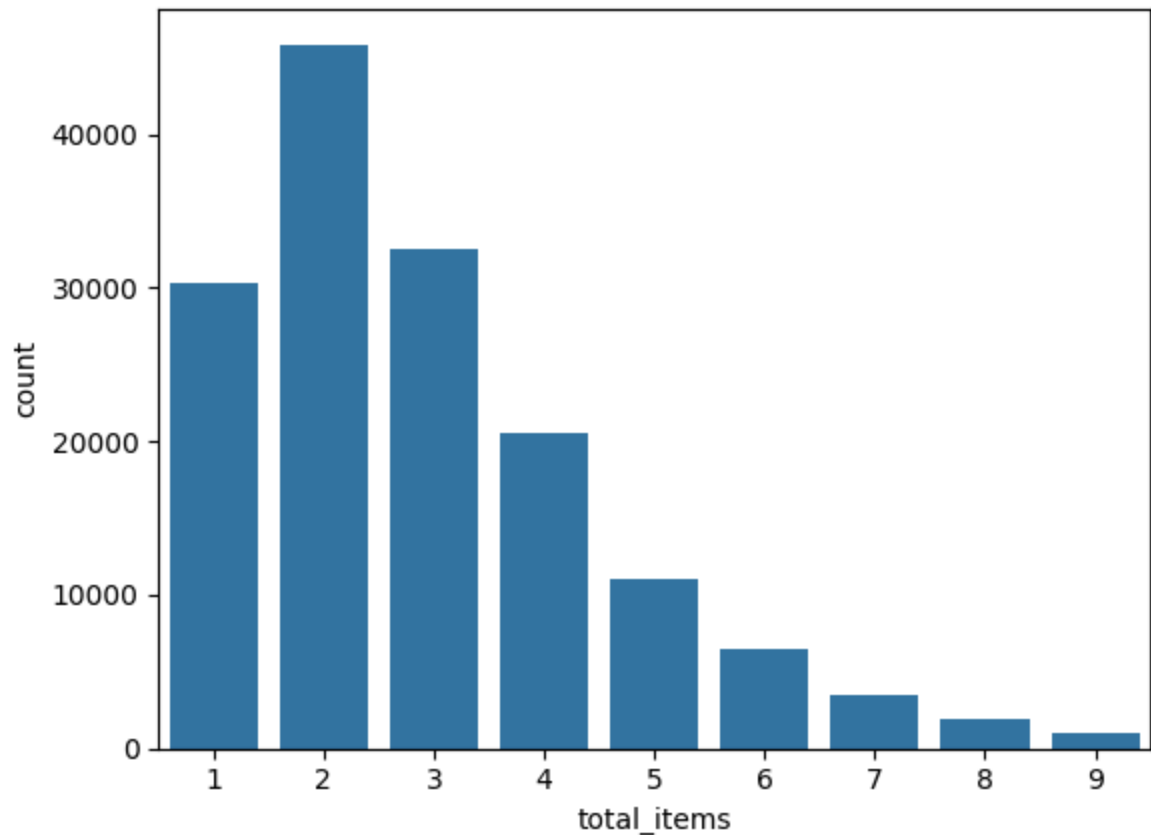

```
In [70]: df['store_primary_category']=df['store_primary_category'].astype('category').cat.co
```

```
In [71]: for col in ['subtotal', 'min_item_price', 'max_item_price', 'delivery_time_mins', '
lower_bound = df[col].quantile(0.01)
upper_bound = df[col].quantile(0.99)
df = df[(df[col] >= lower_bound) & (df[col] <= upper_bound)]
```

```
In [72]: fig, axes = plt.subplots(nrows=4, ncols=2, figsize = (12, 8))
sns.histplot(data=df, x = "subtotal", kde=True, ax=axes[0,0])
sns.boxplot(data=df, x = "subtotal", ax=axes[0,1])
sns.histplot(data=df, x = "min_item_price", kde=True, ax=axes[1,0])
sns.boxplot(data=df, x = "min_item_price", ax=axes[1,1])
sns.histplot(data=df, x = "max_item_price", kde=True, ax=axes[2,0])
sns.boxplot(data=df, x = "max_item_price", ax=axes[2,1])
sns.histplot(data=df, x = "delivery_time_mins", kde=True, ax=axes[3,0])
sns.boxplot(data=df, x = "delivery_time_mins", ax=axes[3,1])
plt.tight_layout()
plt.show()
```



```
In [76]: sns.countplot(data=df, x='total_items')
plt.show()
```

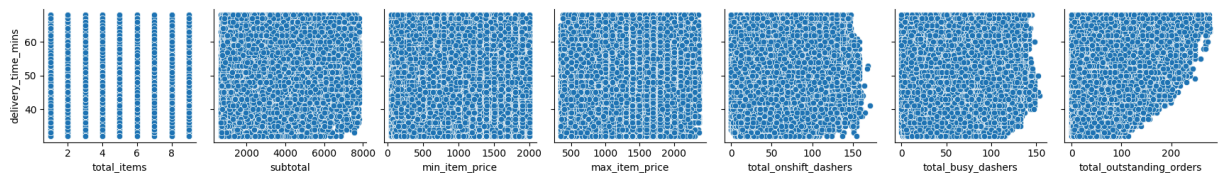


The distribution of data looks better after removing outliers. Most of the delivery is done in 40mins

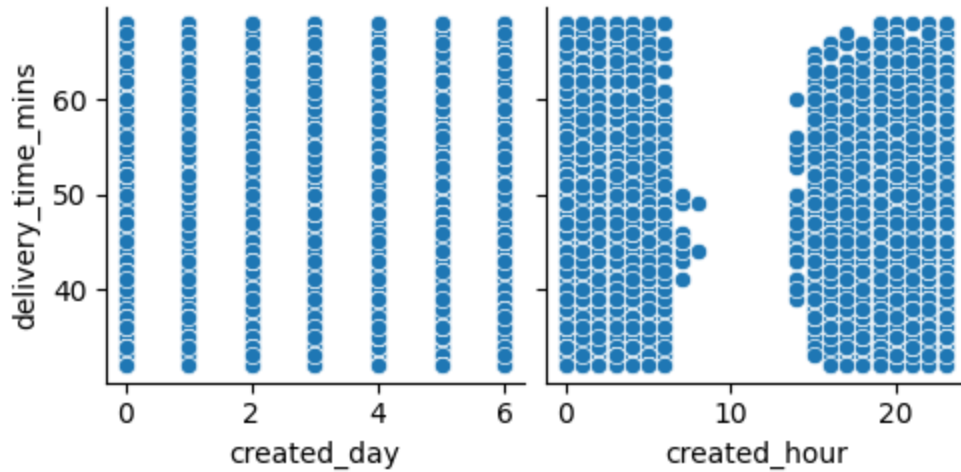
In []:

Bivariate Analysis

```
In [77]: sns.pairplot(
    df,
    x_vars=['total_items', 'subtotal', 'min_item_price', 'max_item_price', 'total_o
    y_vars=['delivery_time_mins'],)
    plt.show()
```



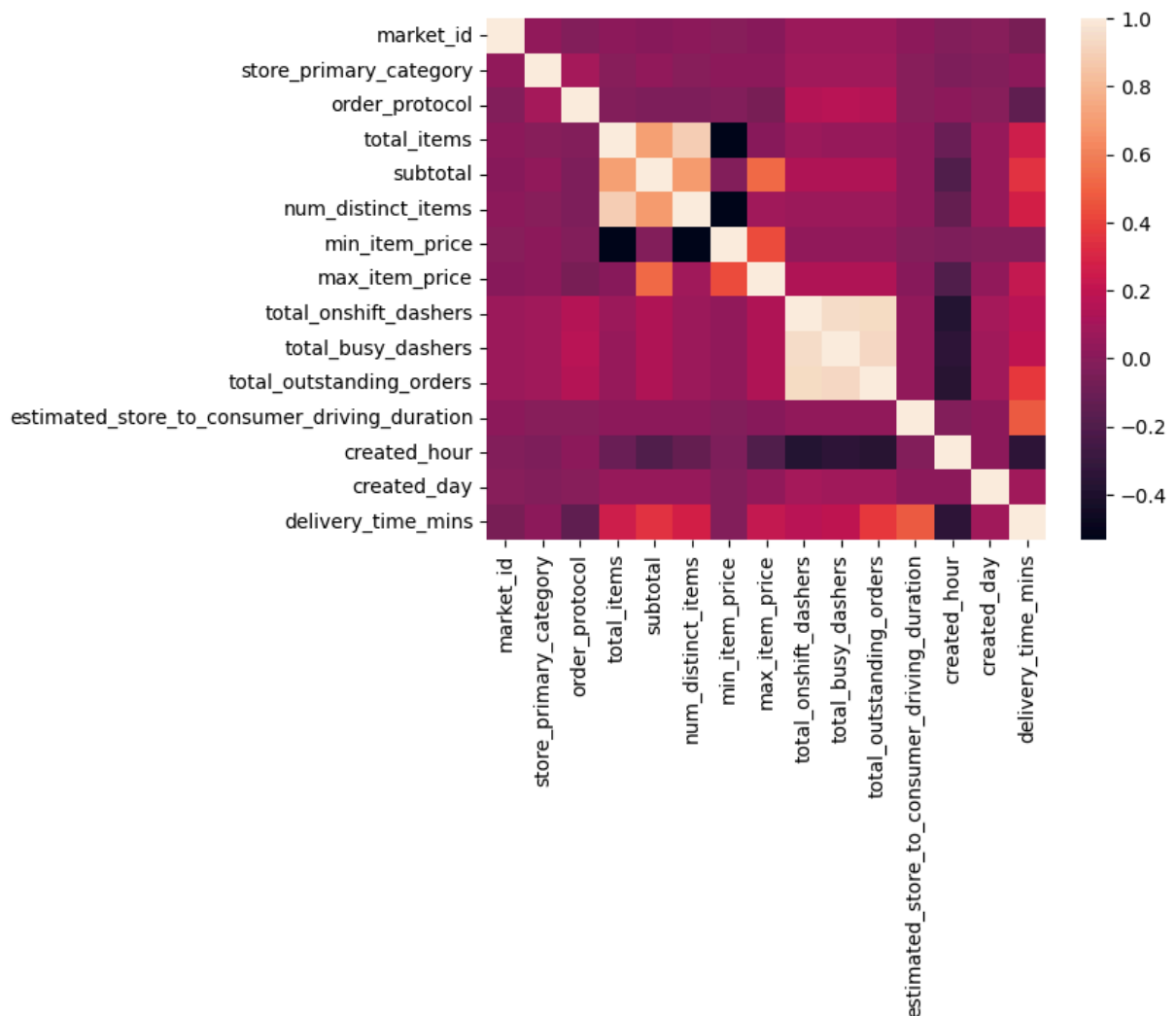
```
In [78]: sns.pairplot(
    df,
    x_vars=['created_day', 'created_hour'],
    y_vars=['delivery_time_mins'],)
    plt.show()
```



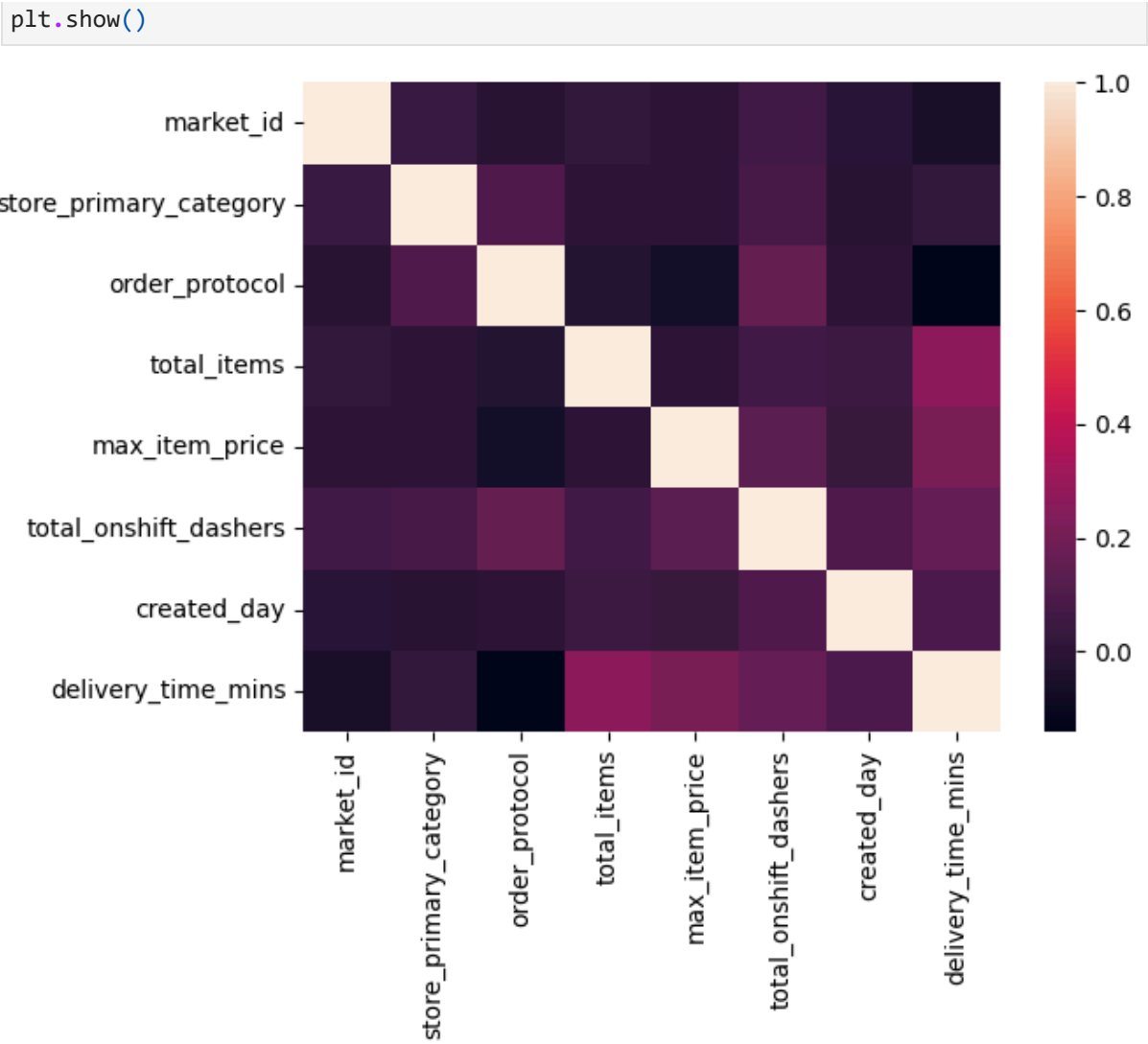
There doesn't seem to be any relationship of delivery time with other features. Delivery time is almost the same for all days.

In []:

```
In [79]: sns.heatmap(df.corr())
plt.show()
```



```
In [80]: sns.heatmap(df[['market_id', 'store_primary_category', 'order_protocol', 'total_ite
```



There is no relation between delivery time and other features

```
In [ ]:
```

NN Modelling

Training a NN model

```
In [81]: df.head()
```

Out[81]:

	market_id	store_primary_category	order_protocol	total_items	subtotal	num_distinct_it
0	1	4	1	4	3441	
1	2	45	2	1	1900	
2	2	35	3	4	4771	
3	1	37	1	1	1525	
4	1	37	1	2	3620	

In [82]: *# Dropping 'store primary category' and 'order protocol' from feature list*

```
df_reduced = df.drop(['store_primary_category', 'order_protocol'], axis=1)
```

In [83]: *# Example to demonstrate categorical encoding*

```
def encode_categorical_data(dataframe):
    # Assume some features are categorical
    categorical_columns = ['market_id']

    # Apply One-Hot Encoding
    one_hot_encoder = OneHotEncoder(sparse_output=False, drop='first') # Drop first
    encoded_features = pd.DataFrame(
        one_hot_encoder.fit_transform(dataframe[categorical_columns]),
        columns=one_hot_encoder.get_feature_names_out(categorical_columns)
    )

    # Drop original categorical columns and concatenate encoded columns
    dataframe = dataframe.drop(categorical_columns, axis=1)
    dataframe = pd.concat([dataframe.reset_index(drop=True), encoded_features.reset_index(drop=True)], axis=1)
    return dataframe
```

In [84]: *# Encode categorical data*

```
df_reduced_encoded = encode_categorical_data(df_reduced)
```

In [85]: df_reduced_encoded.head()

```
Out[85]:
```

	total_items	subtotal	num_distinct_items	min_item_price	max_item_price	total_onshift
0	4	3441	4	557	1239	
1	1	1900	1	1400	1400	
2	4	4771	3	820	1604	
3	1	1525	1	1525	1525	
4	2	3620	2	1425	2195	

In []:

Split and Scale the data

In [86]: *# Split data into features and target*

```
y = df_reduced_encoded['delivery_time_mins']
```

```
X = df_reduced_encoded.drop('delivery_time_mins', axis=1)
```

Split into train and test

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Scale features

```
scaler = MinMaxScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
In [87]: from tensorflow.keras.callbacks import LearningRateScheduler
from keras.layers import LeakyReLU
from tensorflow.keras.losses import LogCosh, Huber
from tensorflow.keras.callbacks import Callback
import tensorflow as tf
from tensorflow.keras.losses import MeanSquaredError
```

NN_Regression model:

Takes input features (X_train) and predicts a continuous numerical value (y_train) which is delivery time

```
In [96]: # Model definition function
def create_and_train_model(X_train, y_train, X_test, y_test, epochs=50):

    # Model definition
    def create_model(input_shape):
        model = Sequential([
            Dense(32, input_shape=(input_shape,)), kernel_initializer='he_normal'),
            BatchNormalization(),
            LeakyReLU(alpha=0.01),
            Dense(64),
            BatchNormalization(),
            LeakyReLU(alpha=0.01),
            Dense(128),
            BatchNormalization(),
            LeakyReLU(alpha=0.01),
            Dense(64),
            BatchNormalization(),
            LeakyReLU(alpha=0.01),
            Dense(1, activation='linear')
        ])

        model.compile(optimizer=Adam(learning_rate=0.001, global_clipnorm=1.0), loss='mse')
        return model

    class GradientLogger(Callback):
        def __init__(self, train_data):
            super().__init__()
            self.train_data = train_data # Training data as (X, y)

        def on_epoch_end(self, epoch, logs=None):
            # Get the training data
            X, y = self.train_data
            with tf.GradientTape() as tape:
                # Forward pass
                y_pred = self.model(X, training=True) # Access self.model directly
                # Compute the loss
                loss = self.model.compiled_loss(y, y_pred)
```

```

# Compute gradients
gradients = tape.gradient(loss, self.model.trainable_variables)

# Log gradient statistics
grad_norms = [tf.norm(g).numpy() for g in gradients if g is not None]
print(f"\nEpoch {epoch + 1}: Gradient Norms:")
print(f"  Mean: {np.mean(grad_norms):.4f}")
print(f"  Min: {np.min(grad_norms):.4f}")
print(f"  Max: {np.max(grad_norms):.4f}")

X_train_sample, y_train_sample = X_train[:100], y_train[:100]
gradient_logger = GradientLogger(train_data=(X_train_sample, y_train_sample))

# Create and train the model
input_shape = X_train.shape[1]
model = create_model(input_shape)
history = model.fit(X_train, y_train, epochs=epochs, verbose=1, validation_spli

# Predictions and final MSE calculation
predictions = model.predict(X_test)
mse = mean_squared_error(y_test, predictions)

# Plot training and validation loss
plt.figure(figsize=(10, 4))
plt.plot(history.history['loss'], label='Training Loss', color='blue')
plt.plot(history.history['val_loss'], label='Validation Loss', color='orange')
plt.xlabel('Epochs')
plt.ylabel('Loss (MSE)')
plt.title('Training and Validation Loss Across Epochs')
plt.legend()
plt.grid(True)
plt.show()

return mse

```

```

In [97]: # Train model with all features
mse_all_features = create_and_train_model(X_train_scaled, y_train, X_test_scaled, y

```

C:\Users\Home\anaconda3\Lib\site-packages\keras\src\layers\core\dense.py:107: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

super().__init__(activity_regularizer=activity_regularizer, **kwargs)
C:\Users\Home\anaconda3\Lib\site-packages\keras\src\layers\activations\leaky_relu.py:41: UserWarning: Argument `alpha` is deprecated. Use `negative\_slope` instead.
warnings.warn(

Epoch 1/50

213/216 ————— 0s 7ms/step - loss: 1709.9170 - mae: 40.7079

Epoch 1: Gradient Norms:

Mean: 42.5064

Min: 0.0000

Max: 351.9182

216/216 ————— 7s 9ms/step - loss: 1705.6023 - mae: 40.6541 - val_loss: 1486.1652 - val_mae: 37.9750

Epoch 2/50

6/216 ————— 2s 10ms/step - loss: 1296.0355 - mae: 35.5363

```
C:\Users\Home\anaconda3\Lib\site-packages\keras\src\backend\tensorflow\trainer.py:69  
6: UserWarning: `model.compiled_loss()` is deprecated. Instead, use `model.compute_l  
oss(x, y, y_pred, sample_weight, training)`.  
    warnings.warn(
```


213/216  0s 9ms/step - loss: 885.1050 - mae: 29.0658
 Epoch 2: Gradient Norms:
 Mean: 45.6871
 Min: 0.0000
 Max: 301.2062

216/216  2s 10ms/step - loss: 880.8814 - mae: 28.9873 - val_loss: 583.4118 - val_mae: 23.8286
 Epoch 3/50

213/216  0s 7ms/step - loss: 186.4107 - mae: 12.0821
 Epoch 3: Gradient Norms:
 Mean: 26.9279
 Min: 0.0000
 Max: 100.0658

216/216  2s 9ms/step - loss: 184.6077 - mae: 11.9864 - val_loss: 51.5025 - val_mae: 6.5871
 Epoch 4/50

215/216  0s 7ms/step - loss: 5.3718 - mae: 1.8213
 Epoch 4: Gradient Norms:
 Mean: 17.5553
 Min: 0.0000
 Max: 60.8930

216/216  2s 8ms/step - loss: 5.3701 - mae: 1.8211 - val_loss: 18.3685 - val_mae: 3.7917
 Epoch 5/50

214/216  0s 8ms/step - loss: 3.2691 - mae: 1.4306
 Epoch 5: Gradient Norms:
 Mean: 10.5439
 Min: 0.0000
 Max: 37.6597

216/216  3s 10ms/step - loss: 3.2675 - mae: 1.4304 - val_loss: 4.0363 - val_mae: 1.6171
 Epoch 6/50

213/216  0s 8ms/step - loss: 2.7688 - mae: 1.3140
 Epoch 6: Gradient Norms:
 Mean: 14.8548
 Min: 0.0000
 Max: 51.3239

216/216  2s 9ms/step - loss: 2.7718 - mae: 1.3148 - val_loss: 3.4180 - val_mae: 1.5370
 Epoch 7/50

209/216  0s 7ms/step - loss: 2.6224 - mae: 1.2822
 Epoch 7: Gradient Norms:
 Mean: 14.9851
 Min: 0.0000
 Max: 52.6832

216/216  2s 8ms/step - loss: 2.6196 - mae: 1.2808 - val_loss: 3.0214 - val_mae: 1.3502
 Epoch 8/50

214/216  0s 7ms/step - loss: 2.4941 - mae: 1.2469
 Epoch 8: Gradient Norms:
 Mean: 13.9484
 Min: 0.0000
 Max: 49.9280

216/216  3s 8ms/step - loss: 2.4950 - mae: 1.2469 - val_loss: 2.6250 - val_mae: 1.3238
 Epoch 9/50

212/216  0s 7ms/step - loss: 2.4247 - mae: 1.2321
 Epoch 9: Gradient Norms:
 Mean: 7.9102
 Min: 0.0000
 Max: 25.9711

216/216  2s 8ms/step - loss: 2.4242 - mae: 1.2320 - val_loss: 2.2621 - val_mae: 1.1890
 Epoch 10/50

212/216  0s 7ms/step - loss: 2.3125 - mae: 1.1995
 Epoch 10: Gradient Norms:
 Mean: 12.4767
 Min: 0.0000
 Max: 42.4929

216/216  2s 9ms/step - loss: 2.3128 - mae: 1.1997 - val_loss: 2.1266 - val_mae: 1.1622
 Epoch 11/50

212/216  0s 7ms/step - loss: 2.2376 - mae: 1.1800
 Epoch 11: Gradient Norms:
 Mean: 7.9756
 Min: 0.0000
 Max: 32.6426

216/216  2s 8ms/step - loss: 2.2347 - mae: 1.1791 - val_loss: 1.9687 - val_mae: 1.0991
 Epoch 12/50

215/216  0s 8ms/step - loss: 2.1811 - mae: 1.1674
 Epoch 12: Gradient Norms:
 Mean: 11.1825
 Min: 0.0000
 Max: 41.8933

216/216  2s 9ms/step - loss: 2.1822 - mae: 1.1677 - val_loss: 2.4313 - val_mae: 1.2837
 Epoch 13/50

214/216  0s 8ms/step - loss: 2.1488 - mae: 1.1550
 Epoch 13: Gradient Norms:
 Mean: 9.8864
 Min: 0.0000
 Max: 38.7424

216/216  2s 9ms/step - loss: 2.1507 - mae: 1.1554 - val_loss: 1.8740 - val_mae: 1.0908
 Epoch 14/50

214/216  0s 8ms/step - loss: 2.1167 - mae: 1.1481
 Epoch 14: Gradient Norms:
 Mean: 9.2573
 Min: 0.0000
 Max: 36.0406

216/216  2s 9ms/step - loss: 2.1167 - mae: 1.1480 - val_loss: 1.8963 - val_mae: 1.0856
 Epoch 15/50

211/216  0s 7ms/step - loss: 2.0870 - mae: 1.1368
 Epoch 15: Gradient Norms:
 Mean: 9.8988
 Min: 0.0000
 Max: 34.6409

216/216  2s 8ms/step - loss: 2.0890 - mae: 1.1376 - val_loss: 1.7027 - val_mae: 1.0211
 Epoch 16/50

211/216  0s 7ms/step - loss: 2.0914 - mae: 1.1409
 Epoch 16: Gradient Norms:
 Mean: 7.1151
 Min: 0.0000
 Max: 24.5100

216/216  2s 8ms/step - loss: 2.1023 - mae: 1.1435 - val_loss: 2.1954 - val_mae: 1.1558
 Epoch 17/50

213/216  0s 8ms/step - loss: 2.0885 - mae: 1.1339
 Epoch 17: Gradient Norms:
 Mean: 9.0830
 Min: 0.0000
 Max: 37.0045

216/216  2s 9ms/step - loss: 2.0871 - mae: 1.1336 - val_loss: 1.7172 - val_mae: 1.0364
 Epoch 18/50

212/216  0s 7ms/step - loss: 2.0315 - mae: 1.1184
 Epoch 18: Gradient Norms:
 Mean: 10.0967
 Min: 0.0000
 Max: 35.4822

216/216  2s 8ms/step - loss: 2.0307 - mae: 1.1183 - val_loss: 2.1737 - val_mae: 1.1881
 Epoch 19/50

215/216  0s 9ms/step - loss: 2.0104 - mae: 1.1134
 Epoch 19: Gradient Norms:
 Mean: 8.2814
 Min: 0.0000
 Max: 32.9600

216/216  2s 10ms/step - loss: 2.0118 - mae: 1.1137 - val_loss: 1.9132 - val_mae: 1.0978
 Epoch 20/50

213/216  0s 7ms/step - loss: 2.0291 - mae: 1.1228
 Epoch 20: Gradient Norms:
 Mean: 7.8088
 Min: 0.0000
 Max: 27.9615

216/216  2s 9ms/step - loss: 2.0286 - mae: 1.1225 - val_loss: 1.7467 - val_mae: 1.0201
 Epoch 21/50

212/216  0s 10ms/step - loss: 2.0192 - mae: 1.1161
 Epoch 21: Gradient Norms:
 Mean: 10.6754
 Min: 0.0000
 Max: 41.1530

216/216  2s 11ms/step - loss: 2.0206 - mae: 1.1162 - val_loss: 1.8784 - val_mae: 1.0656
 Epoch 22/50

216/216  0s 7ms/step - loss: 2.0080 - mae: 1.1132
 Epoch 22: Gradient Norms:
 Mean: 9.9576
 Min: 0.0000
 Max: 38.1871

216/216  2s 8ms/step - loss: 2.0080 - mae: 1.1132 - val_loss: 2.1033 - val_mae: 1.1208
 Epoch 23/50

216/216  0s 7ms/step - loss: 1.9927 - mae: 1.1102
 Epoch 23: Gradient Norms:
 Mean: 8.3013
 Min: 0.0000
 Max: 32.5391

216/216  2s 8ms/step - loss: 1.9927 - mae: 1.1102 - val_loss: 1.7403 - val_mae: 1.0230
 Epoch 24/50

214/216  0s 7ms/step - loss: 1.9664 - mae: 1.1047
 Epoch 24: Gradient Norms:
 Mean: 9.5341
 Min: 0.0000
 Max: 37.9844

216/216  2s 8ms/step - loss: 1.9674 - mae: 1.1050 - val_loss: 1.9152 - val_mae: 1.1095
 Epoch 25/50

214/216  0s 7ms/step - loss: 1.9668 - mae: 1.1038
 Epoch 25: Gradient Norms:
 Mean: 6.6312
 Min: 0.0000
 Max: 24.3376

216/216  2s 8ms/step - loss: 1.9672 - mae: 1.1039 - val_loss: 1.7201 - val_mae: 1.0321
 Epoch 26/50

210/216  0s 8ms/step - loss: 1.9511 - mae: 1.0976
 Epoch 26: Gradient Norms:
 Mean: 6.8391
 Min: 0.0000
 Max: 26.7155

216/216  2s 8ms/step - loss: 1.9489 - mae: 1.0969 - val_loss: 1.6061 - val_mae: 0.9959
 Epoch 27/50

215/216  0s 7ms/step - loss: 1.9326 - mae: 1.0921
 Epoch 27: Gradient Norms:
 Mean: 8.1817
 Min: 0.0000
 Max: 31.8103

216/216  2s 8ms/step - loss: 1.9340 - mae: 1.0925 - val_loss: 1.6989 - val_mae: 1.0410
 Epoch 28/50

213/216  0s 7ms/step - loss: 1.9328 - mae: 1.0938
 Epoch 28: Gradient Norms:
 Mean: 8.2116
 Min: 0.0000
 Max: 31.6324

216/216  2s 8ms/step - loss: 1.9333 - mae: 1.0941 - val_loss: 1.5379 - val_mae: 0.9650
 Epoch 29/50

215/216  0s 7ms/step - loss: 1.9173 - mae: 1.0879
 Epoch 29: Gradient Norms:
 Mean: 6.5018
 Min: 0.0000
 Max: 27.4445

216/216  2s 8ms/step - loss: 1.9191 - mae: 1.0884 - val_loss: 1.8151 - val_mae: 1.0486
 Epoch 30/50

214/216  0s 8ms/step - loss: 1.9221 - mae: 1.0913
 Epoch 30: Gradient Norms:
 Mean: 8.8903
 Min: 0.0000
 Max: 33.8383

216/216  2s 9ms/step - loss: 1.9231 - mae: 1.0916 - val_loss: 1.7958 - val_mae: 1.0526
 Epoch 31/50

212/216  0s 7ms/step - loss: 1.8935 - mae: 1.0826
 Epoch 31: Gradient Norms:
 Mean: 7.4888
 Min: 0.0000
 Max: 30.2100

216/216  2s 8ms/step - loss: 1.8924 - mae: 1.0821 - val_loss: 1.6283 - val_mae: 0.9929
 Epoch 32/50

211/216  0s 7ms/step - loss: 1.8661 - mae: 1.0704
 Epoch 32: Gradient Norms:
 Mean: 8.2654
 Min: 0.0000
 Max: 33.2465

216/216  2s 8ms/step - loss: 1.8697 - mae: 1.0711 - val_loss: 1.7616 - val_mae: 1.0344
 Epoch 33/50

215/216  0s 9ms/step - loss: 1.8928 - mae: 1.0806
 Epoch 33: Gradient Norms:
 Mean: 7.0523
 Min: 0.0000
 Max: 27.0256

216/216  2s 10ms/step - loss: 1.8926 - mae: 1.0805 - val_loss: 1.5914 - val_mae: 0.9947
 Epoch 34/50

212/216  0s 9ms/step - loss: 1.8801 - mae: 1.0775
 Epoch 34: Gradient Norms:
 Mean: 5.8238
 Min: 0.0000
 Max: 24.6707

216/216  2s 10ms/step - loss: 1.8844 - mae: 1.0786 - val_loss: 1.5580 - val_mae: 0.9773
 Epoch 35/50

216/216  0s 9ms/step - loss: 1.8340 - mae: 1.0636
 Epoch 35: Gradient Norms:
 Mean: 6.5512
 Min: 0.0000
 Max: 24.1324

216/216  2s 11ms/step - loss: 1.8340 - mae: 1.0636 - val_loss: 1.5754 - val_mae: 0.9753
 Epoch 36/50

212/216  0s 8ms/step - loss: 1.8778 - mae: 1.0773
 Epoch 36: Gradient Norms:
 Mean: 7.5924
 Min: 0.0000
 Max: 32.2931

216/216  2s 9ms/step - loss: 1.8771 - mae: 1.0771 - val_loss: 1.6471 - val_mae: 1.0023
 Epoch 37/50

215/216  0s 9ms/step - loss: 1.9009 - mae: 1.0810
 Epoch 37: Gradient Norms:
 Mean: 6.2908
 Min: 0.0000
 Max: 27.1417

216/216  2s 10ms/step - loss: 1.9027 - mae: 1.0814 - val_loss: 1.5506 - val_mae: 0.9773
 Epoch 38/50

213/216  0s 9ms/step - loss: 1.8429 - mae: 1.0663
 Epoch 38: Gradient Norms:
 Mean: 7.3051
 Min: 0.0000
 Max: 30.2210

216/216  2s 10ms/step - loss: 1.8449 - mae: 1.0668 - val_loss: 1.7585 - val_mae: 1.0162
 Epoch 39/50

214/216  0s 9ms/step - loss: 1.8440 - mae: 1.0660
 Epoch 39: Gradient Norms:
 Mean: 6.9521
 Min: 0.0000
 Max: 26.2890

216/216  2s 10ms/step - loss: 1.8451 - mae: 1.0663 - val_loss: 1.6267 - val_mae: 1.0058
 Epoch 40/50

211/216  0s 9ms/step - loss: 1.8375 - mae: 1.0629
 Epoch 40: Gradient Norms:
 Mean: 6.8075
 Min: 0.0000
 Max: 28.7213

216/216  2s 10ms/step - loss: 1.8461 - mae: 1.0656 - val_loss: 1.6649 - val_mae: 1.0037
 Epoch 41/50

213/216  0s 10ms/step - loss: 1.8705 - mae: 1.0723
 Epoch 41: Gradient Norms:
 Mean: 4.9086
 Min: 0.0000
 Max: 21.3019

216/216  2s 11ms/step - loss: 1.8725 - mae: 1.0731 - val_loss: 1.5469 - val_mae: 0.9760
 Epoch 42/50

212/216  0s 11ms/step - loss: 1.8457 - mae: 1.0682
 Epoch 42: Gradient Norms:
 Mean: 4.8519
 Min: 0.0000
 Max: 19.6628

216/216  3s 12ms/step - loss: 1.8467 - mae: 1.0688 - val_loss: 1.6072 - val_mae: 0.9944
 Epoch 43/50

215/216  0s 9ms/step - loss: 1.8242 - mae: 1.0588
 Epoch 43: Gradient Norms:
 Mean: 5.9756
 Min: 0.0000
 Max: 24.3422

216/216  5s 10ms/step - loss: 1.8253 - mae: 1.0591 - val_loss: 1.7088 - val_mae: 1.0173
 Epoch 44/50

210/216  0s 9ms/step - loss: 1.8280 - mae: 1.0585
 Epoch 44: Gradient Norms:
 Mean: 5.5779
 Min: 0.0000
 Max: 24.4848

216/216  2s 10ms/step - loss: 1.8272 - mae: 1.0580 - val_loss: 1.6563 - val_mae: 1.0074
 Epoch 45/50

214/216  0s 9ms/step - loss: 1.8683 - mae: 1.0719
 Epoch 45: Gradient Norms:
 Mean: 5.1941
 Min: 0.0000
 Max: 20.3159

216/216  2s 10ms/step - loss: 1.8704 - mae: 1.0725 - val_loss: 1.9154 - val_mae: 1.0823
 Epoch 46/50

215/216  0s 8ms/step - loss: 1.8232 - mae: 1.0609
 Epoch 46: Gradient Norms:
 Mean: 6.6090
 Min: 0.0000
 Max: 28.3864

216/216  2s 9ms/step - loss: 1.8237 - mae: 1.0609 - val_loss: 1.5900 - val_mae: 0.9828
 Epoch 47/50

214/216  0s 9ms/step - loss: 1.8058 - mae: 1.0525
 Epoch 47: Gradient Norms:
 Mean: 6.3618
 Min: 0.0000
 Max: 28.3245

216/216  2s 10ms/step - loss: 1.8115 - mae: 1.0541 - val_loss: 1.5997 - val_mae: 0.9882
 Epoch 48/50

215/216  0s 10ms/step - loss: 1.8037 - mae: 1.0525
 Epoch 48: Gradient Norms:
 Mean: 6.7195
 Min: 0.0000
 Max: 29.6540

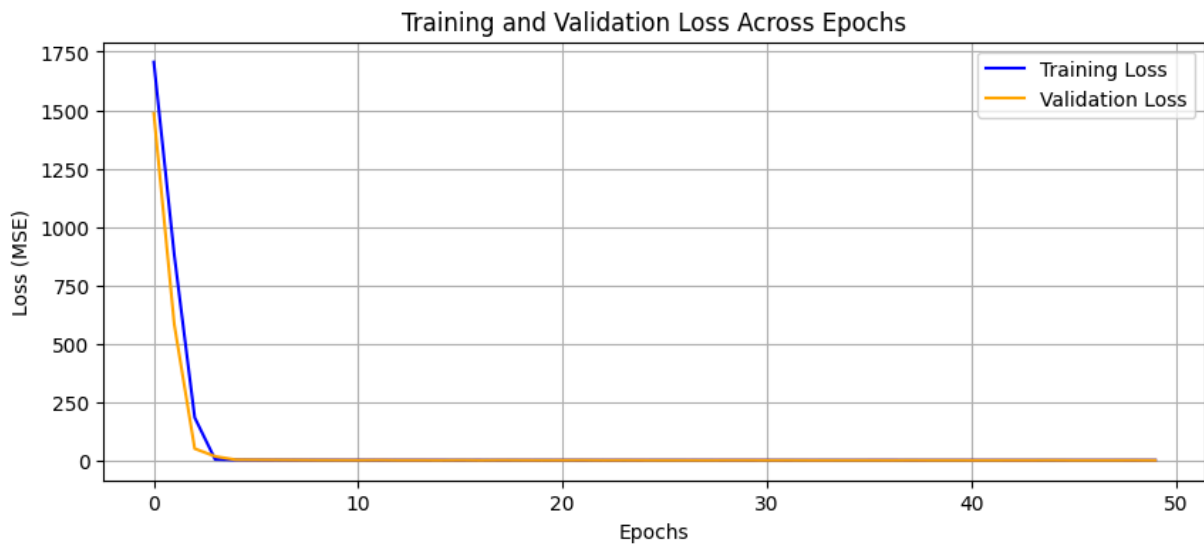
216/216  2s 11ms/step - loss: 1.8051 - mae: 1.0528 - val_loss: 1.6037 - val_mae: 0.9797
 Epoch 49/50

211/216  0s 9ms/step - loss: 1.8287 - mae: 1.0600
 Epoch 49: Gradient Norms:
 Mean: 7.0180
 Min: 0.0000
 Max: 28.3437

216/216  2s 10ms/step - loss: 1.8307 - mae: 1.0612 - val_loss: 1.7347 - val_mae: 1.0677
 Epoch 50/50

211/216  0s 9ms/step - loss: 1.8228 - mae: 1.0594
 Epoch 50: Gradient Norms:
 Mean: 6.8097
 Min: 0.0000
 Max: 30.3872

216/216  2s 10ms/step - loss: 1.8272 - mae: 1.0604 - val_loss: 1.5262 - val_mae: 0.9706
957/957  3s 2ms/step



In [98]: mse_all_features

Out[98]: np.float64(1.5409664325660857)

In []:

- The training and validation errors remained relatively close, so there was no strong evidence of severe overfitting in the final training run. Training Validation MSE 1.8272 1.5262 MAE 1.0604 0.9706 - For epoch 50: Mean = 6.8097 Min = 0.0000 Max = 30.3872 which means Gradients aren't exploding throughout training. - The NN_regression model achieved a validation MSE of 1.5262 and MAE of 0.9706, indicating an average prediction error of approximately 0.97 units of delivery time.

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []: